



I118 - Taller de Proyecto II

Informe de Avance - Octubre

Proyecto A5 - Mano simulada DMP

Grupo de desarrollo

Acuña, Lucia - 03213/1

Avila Montoya, Eygleen Fernanda - 02931/2

Bejarano, Abril - 03339/5

1. Descripción general del proyecto

El proyecto tiene como objetivo comparar dos métodos de estimación de orientación tridimensional. Para esto, se deberá desarrollar un sistema de adquisición y visualización de orientación a partir de los datos obtenidos de una unidad de medición inercial de 9 ejes. El primer método utilizará la orientación calculada por el procesador digital de movimiento (DMP) del sensor y el segundo empleará filtros implementados por software sobre las mediciones del acelerómetro, el giroscopio y el magnetómetro.

El dispositivo final estará compuesto por un módulo GY-9250 con sensor MPU9250 montado sobre un guante y conectado por I²C a un ESP32. Los resultados se transmitirán a una interfaz web que mostrará dos modelos tridimensionales y una gráfica de diferencias entre estimaciones.

Se decidió utilizar el MPU9250 en lugar del MPU6050 inicialmente considerado para incorporar también un magnetómetro, potencialmente útil como referencia del eje de yaw. En esta primera etapa se utilizó un MPU6050 para verificar adquisición, interrupciones, calibración inicial y filtrado, por lo que los resultados experimentales de octubre corresponden éste sensor.

2. Materiales y presupuesto

Tabla 1: Materiales necesarios para el desarrollo del prototipo y su disponibilidad.

Cantidad	Descripción	Disponible	Precio unitario
1	ESP32 NodeMCU WROOM-32, 38 pines, USB-C, Wi-Fi y Bluetooth	Sí	$20.700
1	Módulo GY-9250 con MPU9250: acelerómetro, giroscopio y magnetómetro; DMP	No	$21.960
1	MPU6050 empleado para las pruebas preliminares	Sí	$6.000
1	Magnetómetro QMC5883L empleado para las pruebas preliminares	Sí	$7.266
1	Guante para montaje del sistema	Pendiente	A definir
1	Juego de cables Dupont x10	Sí	—
1	Capacitor de desacople de 0,1 µF	Sí	—
1	Cable MicroUSB para alimentación y programación	Sí	—
1	Power bank para alimentación portátil (opcional)	Pendiente	A definir
Total			**$48.660**

La cátedra proporcionó el ESP32 y dos unidades del módulo GY-9250. El MPU6050 y el magnetómetro QMC5883L se emplearon únicamente como dispositivos de ensayo durante la

etapa preliminar. Los materiales restantes ya se encontraban disponibles, por lo que solo resta adquirir el guante sobre el cual se montará el sistema.

En la propuesta inicial se había contemplado el uso de tres capacitores de desacople. Sin embargo, al realizar el montaje se comprobó que dos de ellos correspondían a pines que no se utilizan en este proyecto, por lo que la cantidad se redujo a uno. Esta decisión deberá verificarse al montar el equipo final, ya que existe la posibilidad de que no se requiera ninguno.

Asimismo, se había previsto un cable USB-C para la alimentación, pero el ESP32 provisto por la cátedra requiere un cable MicroUSB, por lo que se lo reemplazó.

El resto de los materiales se mantienen según lo establecido en el plan de proyecto. Los costos de los dispositivos empleados en las pruebas preliminares son afrontados por el grupo de desarrollo, dado que son externos al prototipo final y, por lo tanto, no se incluyen en el total del presupuesto.

3. Esquema gráfico del proyecto

La arquitectura propuesta separa las etapas de adquisición, procesamiento y visualización. El ESP32 recibe las mediciones por I²C, estima la orientación mediante los dos métodos previstos y transmite los resultados a la interfaz de visualización. Este esquema, que se muestra en la Figura 1, coincide con el presentado en el Plan de Proyecto, ya que no se realizaron modificaciones.

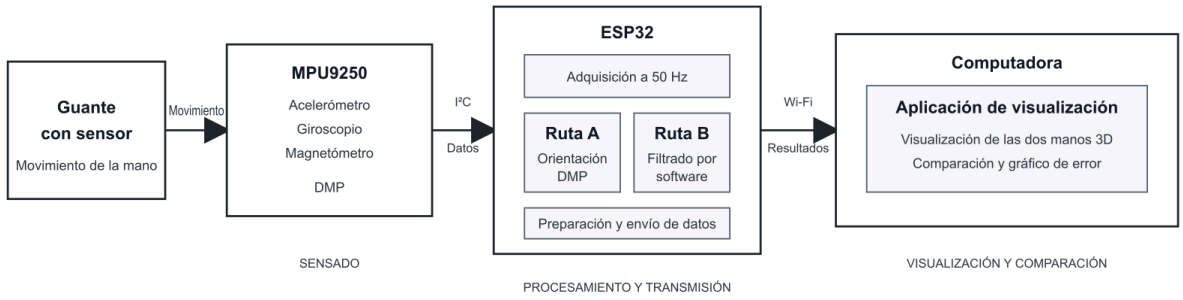


Figura 1: Diagrama de bloques del sistema presentado en el Plan de Proyecto.

Durante octubre se validó una ruta provisional que reemplaza el enlace Wi-Fi por una conexión USB serie con la computadora, mientras que la transmisión inalámbrica permanece sin verificar. Asimismo, en esta etapa se emplearon el MPU6050 y el magnetómetro QMC5883L en lugar del MPU9250, y la estimación se limitó al filtro complementario ejecutado en el ESP32. El esquema del prototipo probado a la fecha se presenta en la Figura 2.

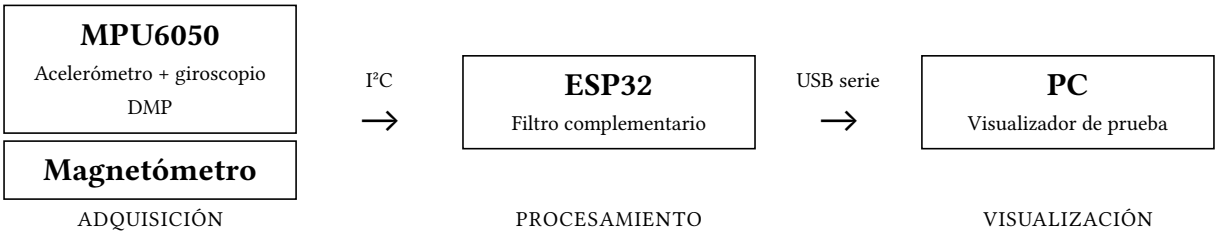
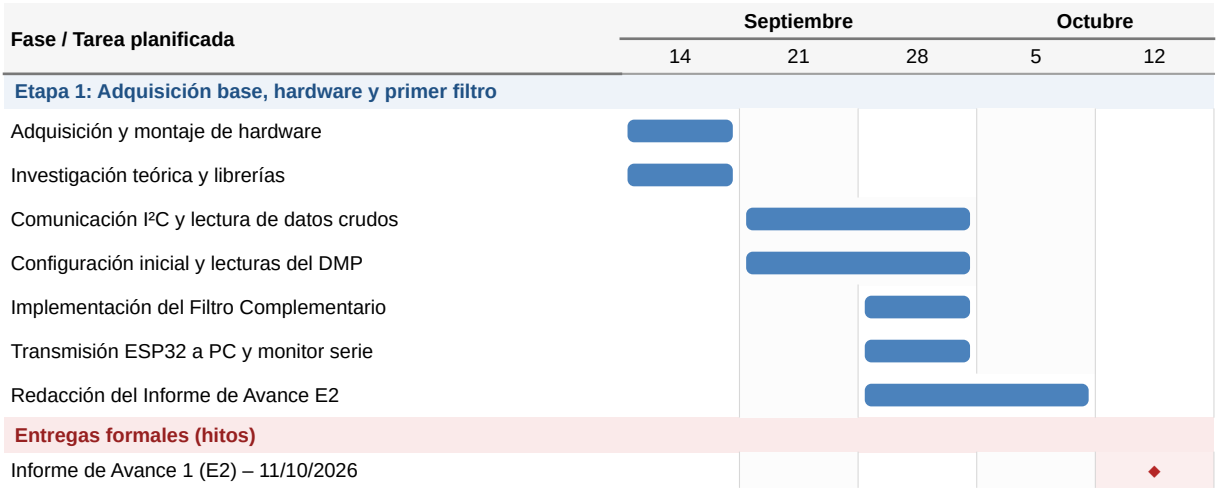


Figura 2: Ruta experimental validada en octubre, hasta la transmisión por USB serie.

4. Grado de avance, problemas y soluciones

En el plan de proyecto se planificó para esta etapa completar la adquisición y montaje del hardware, establecer la comunicación I²C con el sensor y WiFi con la computadora, implementar el filtro complementario y realizar las lecturas del DMP aunque sin realizar configuraciones ni calibraciones por ahora. El diagrama de Gantt de la Tabla 2 muestra esta planificaci;on original.

Tabla 2: Diagrama de Gantt para la entrega de octubre.



En las siguientes secciones se evalúa el grado de avance de cada uno de estos items.

4.1. Montaje de hardware

4.1.1. Conexión y alimentación

Se armó un prototipo con un ESP32, un MPU6050 y un magnetómetro QMC5883L. Los conductores se soldaron directamente a los pines de los módulos y se conectaron al ESP32, que se alimentó por USB y suministró 3,3 V a ambos sensores. Las conexiones realizadas se detallan en la Tabla 3.

Tabla 3: Conexiones verificadas del prototipo experimental.

QMC5883L	MPU6050	ESP32	Función
VCC	VCC	3V3	Alimentación
GND	GND	GND	Masa común
SDA	SDA	GPIO21	Datos I²C
SCL	SCL	GPIO22	Reloj I²C
	INT	GPIO23	Interrupción DATA_READY
	AD0	—	Selección de dirección en el módulo

Ambos sensores comparten los pines SDA y SCL, ya que se comunican por el mismo bus I²C. El pin AD0 del MPU6050 define su registro de identificación: conectado a GND se selecciona

la dirección `0x68`, y conectado a VCC, la `0x69`. Durante las pruebas se comprobó que el módulo ya trae AD0 conectado a tierra, por lo que se lo dejó sin conectar y el sensor responde en `0x68`.

El pin INT se utiliza para indicar que hay nuevas mediciones disponibles (señal DATA_READY), lo que permite adquirir los datos por interrupción en lugar de por sondeo. Los pines DRDY del magnetómetro y los pines XDA y XCL del MPU6050 no se conectaron.

En la Figura 3 se muestra gráficamente la conexión descripta.

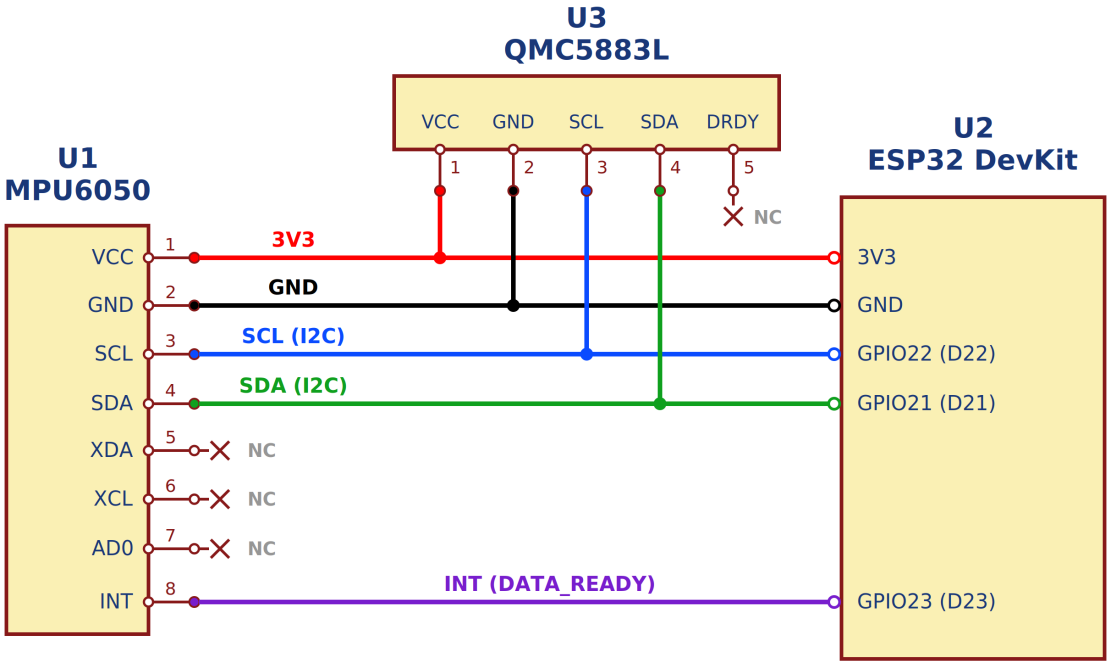


Figura 3: Esquema de arquitectura general del sistema.

Los requisitos de alimentación son los previstos en el Plan de Proyecto. Se resumen en la Tabla 4.

Tabla 4: Requerimientos de alimentación de los componentes del prototipo final.

Componente	**Condición de operación**	**Alimentación**	**Corriente**
ESP32-WROOM-32	CPU a 50 MHz	3,3 V	< 25 mA
ESP32-WROOM-32	Módulo Wi-Fi a 1 Mbps	3,3 V	240 mA
ESP32-WROOM-32	Bluetooth	—	No se utiliza
MPU9250	Giroscopio a 1 kHz, acelerómetro a 4 kHz y magnetómetro a 8 Hz, sin DMP (máximo)	VDD: 2,4 a 3,6 V VDDIO: 1,71 V a VDD	3,7 mA
MPU9250	9 ejes y DMP habilitados (aproximado, si los sensores no están al máximo)	VDD: 2,4 a 3,6 V VDDIO: 1,71 V a VDD	3,5 mA
Consumo total estimado del ESP32 (CPU + Wi-Fi)			**< 265 mA**

4.2. Investigación teórica y librerías

Se estudiaron los fundamentos de la adquisición con acelerómetro y giroscopio, incluyendo los rangos de sensibilidad, las interrupciones DATA_READY y el funcionamiento del DMP. Como primer paso se utilizó la biblioteca `Adafruit_MPU6050` para comprobar la lectura de aceleración, velocidad angular y temperatura. Luego se pasó al acceso directo a los registros del sensor mediante la librería `Wire`, lo que permite controlar la configuración y la lectura sin depender de una biblioteca específica del dispositivo.

Para las pruebas del DMP se empleó la librería I2Cdevlib, propuesta en el artículo «MPU6050 Gyro Drift Fix: DMP, Kalman Filter, and I2Cdevlib Setup» para obtener la orientación calculada por el DMP del sensor y reducir así la deriva del giroscopio. Se utilizó el archivo de cabecera `MPU6050_6Axis_MotionApps20.h`, que incluye el firmware del DMP que debe cargarse en el sensor durante la inicialización. Este firmware corresponde al MPU6050, por lo que deberá verificarse una alternativa equivalente para el MPU9250, cuyo DMP requiere un firmware y una forma de carga distintos.

Para la visualización se investigó la librería Three.js, una biblioteca de JavaScript basada en WebGL que permite crear y renderizar escenas tridimensionales directamente en el navegador, sin necesidad de instalar software adicional. Se estudiaron sus elementos básicos y la forma de representar rotaciones mediante cuaterniones y ángulos de Euler, lo que resulta adecuado para mostrar la orientación estimada. Con ella se desarrolló un visualizador de prueba que reproduce en tiempo real la orientación del sensor sobre un modelo tridimensional. Además, al ser una tecnología web, es la misma que se empleará en la interfaz final para mostrar los dos modelos de la mano y comparar ambas estimaciones.

4.3. Comunicación I²C y lectura de datos crudos

La comunicación con los sensores se realizó mediante la librería `Wire`, con SDA en GPIO21, SCL en GPIO22 y el bus a 100 kHz. Se implementaron dos funciones de acceso a registros: una de escritura (Código 1) y otra de lectura de uno o más bytes consecutivos (Código 2). Esta última emplea un reinicio repetido entre el envío de la dirección del registro y la lectura, y se limitó a 32 bytes por transacción.

```
1 int i2c_escribir_registro(uint8_t direccion,
2                           uint8_t registro,
3                           uint8_t valor){
4     Wire.beginTransmission(direccion);
5     Wire.write(registro);
6     Wire.write(valor);
7     return Wire.endTransmission() == 0 ? 0 : -1;
8 }
```

Código 1: Función de lectura con I2C.

```
1  int i2c_leer_registros(uint8_t direccion,
2                         uint8_t registro,
3                         uint8_t* buffer,
```

```
4                       size_t longitud){
5       if (buffer == nullptr || longitud == 0 || longitud > 32){
6           return -1;
7       }
8       Wire.beginTransmission(direccion);
9       Wire.write(registro);
10      if (Wire.endTransmission(false) != 0){
11          return -1;
12      }
13      size_t recibidos = Wire.requestFrom(
14          direccion , longitud , true
15      );
16      if (recibidos != longitud){
17          return -1;
18      }
19      for (size_t i = 0; i < longitud; i++){
20          buffer[i] = Wire.read();
21      }
22      return 0;
23  }
```

Código 2: Función de lectura con I2C.

El MPU6050 (dirección `0x68`) se sacó del modo de suspensión y se seleccionó como fuente de reloj el PLL referido al giroscopio del eje X. Se configuró el acelerómetro en ±2 g, el giroscopio en ±250 °/s y el DLPF en aproximadamente 21 Hz (21 Hz para el acelerómetro y 20 Hz para el giroscopio). Con el filtro activado, el divisor de frecuencia de muestreo se fijó en 19, lo que da 1 kHz / (1 + 19) = 50 Hz (Código 3). Cada vez que se realiza una lectura se leen 14 bytes consecutivos a partir del registro `0x3B`, correspondientes a aceleración, temperatura y velocidad angular. Los valores enteros de 16 bits se convirtieron a m/s² y rad/s con las sensibilidades de 16.384 LSB/g y 131 LSB/(°/s), y la temperatura a °C (Código 4).

```
1   int mpu_init(void){
2       uint8_t id = 0;
3       if (i2c_leer_registros(MPU_ADDR , WHO_AM_I , &id , 1) != 0)return -1;
4       mpu_id = id;
5       if (i2c_escribir_registro(MPU_ADDR , PWR_MGMT_1 , 0x01) != 0)return -1;
6       // 1 kHz / (1 + 19) = 50 Hz (con DLPF activado).
7       if (i2c_escribir_registro(MPU_ADDR , SMPLRT_DIV , 19) != 0)return -1;
8       if (i2c_escribir_registro(MPU_ADDR , CONFIG , 0x04) != 0)return -1;
9       if (i2c_escribir_registro(MPU_ADDR , GYRO_CONFIG , 0x00) != 0)return -1;
10      if (i2c_escribir_registro(MPU_ADDR , ACCEL_CONFIG , 0x00) != 0)return -1;
11      printf("MPU configurado (+/-2g, +/-250deg/s, DLPF 21Hz, 50Hz).\n");
12      return 0;
13  }
```

Código 3: Configuración del MPU6050.

```
int mpu_leer(mpu_datos_t* d){
  if (!d || !mpu_id)return -1;
  uint8_t b[14];
  if (i2c_leer_registros(MPU_ADDR , ACCEL_XOUT_H , b , sizeof(b))
                        != 0)return -1;
  d->ax = (a16(b[0] , b[1]) / 16384.0f) * G;
  d->ay = (a16(b[2] , b[3]) / 16384.0f) * G;
  d->az = (a16(b[4] , b[5]) / 16384.0f) * G;
  d->temperatura = a16(b[6] , b[7]) / 340.0f + 36.53f;
  const float conv = (PI_F / 180.0f) / 131.0f;
  d->gx = a16(b[8] , b[9]) * conv;
  d->gy = a16(b[10] , b[11]) * conv;
  d->gz = a16(b[12] , b[13]) * conv;
  return 0;
}
```

Código 4: Lectura de acelerómetro, giroscopio y temperatura del MPU.

Para el magnetómetro se implementó una detección automática durante la inicialización. Se consulta primero la dirección `0x0D` (QMC5883L) y, si no responde, la `0x1E` (HMC5883L), y en ambos casos se configura el modo de medición continua. Se configuró el sensor con rango de ±8 G y sobremuestreo de 512. Cada lectura obtiene 6 bytes, el QMC5883L entrega los datos desde el registro `0x00` en formato little-endian y orden X, Y, Z (Código 5). Los valores se mantienen como cuentas crudas del conversor, sin conversión a unidades físicas. La calibración de hard-iron quedó pendiente, con los offsets de cada eje en cero.

```
int mag_leer(mag_datos_t* d) {
  if (!d || mag_addr == 0) return -1;
  uint8_t b[6];
  if (es_qmc) {
    // QMC5883L: registros 0x00..0x05, Little-Endian, orden X, Y, Z
    if (i2c_leer_registros(mag_addr, 0x00, b, 6) != 0) return -1;
    int16_t x = (int16_t)((b[1] << 8) | b[0]);
    int16_t y = (int16_t)((b[3] << 8) | b[2]);
    int16_t z = (int16_t)((b[5] << 8) | b[4]);
    d->mx = (float)x;
    d->my = (float)y;
    d->mz = (float)z;
  }
  ...
}
```

Código 5:

La sincronización se realizó con la interrupción DATA_READY del MPU6050 en GPIO23, con una frecuencia nominal de 50 Hz. Como el pin DRDY del magnetómetro no está conectado, este se consulta en el ciclo marcado por dicha interrupción.

En una prueba de 1500 muestras realizada antes de incorporar el magnetómetro, se registró una frecuencia media de 50,090 Hz, sin errores I²C como se detalla a continuación.

4.3.1. Calibración inicial y ruido en reposo

Se adquirieron 1500 muestras con el MPU6050 inmóvil durante 29,926 segundos, calculando medias y desviaciones estándar muestrales por eje. La prueba permitió estimar el sesgo inicial del giroscopio y observar la variabilidad del acelerómetro.

Tabla 5: Mediciones del acelerómetro en reposo.

Eje	Media (m/s²)	Desv. estándar (m/s²)
Ax	−0,028250	0,010691
Ay	0,064062	0,009362
Az	9,935172	0,016313

Tabla 6: Sesgos preliminares y dispersión del giroscopio en reposo.

Eje	Media (rad/s)	Desv. estándar (rad/s)
Gx	−0,022842	0,000306
Gy	0,000215	0,000408
Gz	−0,006985	0,000341

Las medias del giroscopio se utilizaron como estimaciones preliminares de su bias. Si el valor medido en X (−0,022842 rad/s) permaneciera constante, su integración sin corrección produciría una deriva teórica de aproximadamente 78,5° en un minuto. En el acelerómetro, una única posición de reposo no permite separar errores de escala, alineación y offset.

4.4. Configuración inicial y lecturas del DMP

Se desarrolló un programa de prueba para la ruta A (orientación calculada por el DMP) sobre el MPU6050, utilizando la librería I2Cdevlib. El programa configura el bus I²C a 400 kHz, para acelerar la lectura de la FIFO, e inicializa el sensor. Luego carga el firmware del DMP con `dmpInitialize()`, establece offsets iniciales de acelerómetro y giroscopio, y ejecuta una calibración automática (`CalibrateAccel` y `CalibrateGyro`). Finalmente habilita el DMP y la interrupción en GPIO23, que se activa cuando hay un paquete disponible en la FIFO. (Código 6)

```
1  Wire.begin(21, 22);
2  Wire.setClock(400000);
3
4  mpu.initialize();
5
6  devStatus = mpu.dmpInitialize();
7
8  mpu.setXAccelOffset(-1000);
9  mpu.setYAccelOffset(-1000);
10 mpu.setZAccelOffset(1000);
```

```
11 mpu.setXGyroOffset(0);
12 mpu.setYGyroOffset(0);
13 mpu.setZGyroOffset(0);
14
15 if (devStatus == 0) {
16     mpu.CalibrateAccel(6);
17     mpu.CalibrateGyro(6);
18     mpu.setDMPEnabled(true);
19     attachInterrupt(digitalPinToInterrupt(PIN_INT), isrDmp, RISING);
20     mpuIntStatus = mpu.getIntStatus();
21     dmpListo = true;
22     packetSize = mpu.dmpGetFIFOPacketSize();
23     Serial.println("DMP inicializado con exito.");
24 }
```

Código 6: Inicialización del sensor, calibración y hablitacion del DMP.

En cada ciclo se lee el paquete de la FIFO y se extrae el cuaternión de orientación, a partir del cual se obtiene el vector de gravedad y los ángulos de yaw, pitch y roll. Estos ángulos se convierten a grados y se envían por el puerto serie a 10 Hz (Código 7).

```
1  if (mpu.dmpGetCurrentFIFOPacket(fifoBuffer)) {
2      mpu.dmpGetQuaternion(&q, fifoBuffer);
3      mpu.dmpGetGravity(&gravity, &q);
4      mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);
5      static uint32_t ultimoPrint = 0;
6      if (millis() - ultimoPrint >= 100) { // 10 Hz
7          ultimoPrint = millis();
8          Serial.printf("DMP -> R: %+6.1f° | P: %+6.1f° | Y: %+6.1f°\n",
9          ypr[2]*180.0f / M_PI,ypr[1]*180.0f / M_PI,ypr[0]*180.0f / M_PI);
10     }
11 }
```

Código 7: Lectura de un paquete de la FIFO.

Por lo tanto, ya se dispone de una lectura de orientación proveniente del DMP del MPU6050, aunque solo se comprobó su funcionamiento de forma cualitativa y no se registraron su frecuencia, su estabilidad ni su deriva.

Esta prueba se realizó con el MPU6050 y no valida el DMP del MPU9250, cuyo firmware y forma de carga son distintos. Además, el DMP del MPU6050 calcula la orientación a partir de acelerómetro y giroscopio únicamente, por lo que el yaw carece de referencia absoluta y es esperable que presente deriva.

Como próximos pasos se prevé conectar el GY-9250, comprobar su identificación, configurar la lectura de orientación del DMP y registrar su frecuencia y estabilidad.

4.5. Implementación del filtro complementario

Se implementó en el ESP32 un filtro complementario que estima la orientación en roll, pitch y yaw combinando la integración del giroscopio con dos referencias absolutas: el acelerómetro, que da la inclinación respecto de la gravedad, y el magnetómetro, que da el rumbo. En los tres ejes se adoptó un coeficiente de ponderación $\alpha = 0,98$, de modo que el giroscopio domina en el corto plazo y las referencias corrigen la deriva en el largo plazo. Antes de integrar, se restan a las velocidades angulares los sesgos medidos con el sensor en reposo (Código 8).

```
#define ALPHA 0.98f
// Bias del giroscopio medido en reposo (rad/s)
#define BIAS_GX -0.022842f
#define BIAS_GY  0.000215f
#define BIAS_GZ -0.006985f
// Calibración inicial simple (offset / hard-iron) del magnetómetro
#define MAG_OFFSET_X 0.0f
#define MAG_OFFSET_Y 0.0f
#define MAG_OFFSET_Z 0.0f
static Orientacion orientacion = {0.0f, 0.0f, 0.0f};
```

Código 8: Parámetros del filtro y calibración.

El procesamiento se organizó en los archivos `filtro_complementario.cpp` y `filtro_complementario.h`, separados de los módulos de adquisición. La estimación se inicializa con la primera muestra (Código 9): roll y pitch se obtienen del acelerómetro y el yaw del magnetómetro, de modo que el filtro parte de una orientación coherente con la posición real del sensor.

```
void filtro_iniciar(const mpu_datos_t &d, const mag_datos_t &m) {
  orientacion.roll = atan2f(d.ay, d.az);
  orientacion.pitch = atan2f(-d.ax, sqrtf(d.ay * d.ay + d.az * d.az));
  orientacion.yaw = calcular_yaw_mag(m, orientacion.roll, orientacion.pitch);
}
```

Código 9: Inicialización con la primera muestra.

Luego, la actualización se ejecuta en cada nueva muestra y utiliza el intervalo de tiempo transcurrido desde la anterior, descartando los valores no positivos o no finitos de este intervalo (Código 10).

```
Orientacion filtro_actualizar(const mpu_datos_t &d, const mag_datos_t &m, float dt) {

  if (dt <= 0.0f || !isfinite(dt)) {
      return orientacion;
  }

  // 1. Corregir bias del giroscopio
  const float gx = d.gx - BIAS_GX;
  const float gy = d.gy - BIAS_GY;
```

```
10    const float gz = d.gz - BIAS_GZ;
11
12    // 2. Inclinación con acelerómetro
13    const float roll_acc = atan2f(d.ay, d.az);
14    const float pitch_acc = atan2f(-d.ax, sqrtf(d.ay * d.ay + d.az * d.az));
15
16    // 3. Integración con giroscopio
17    const float roll_gyro = orientacion.roll + gx * dt;
18    const float pitch_gyro = orientacion.pitch + gy * dt;
19    const float yaw_gyro = orientacion.yaw + gz * dt;
20
21    // 4. Fusión Roll y Pitch
22    orientacion.roll = ALPHA * roll_gyro + (1.0f - ALPHA) * roll_acc;
23    orientacion.pitch = ALPHA * pitch_gyro + (1.0f - ALPHA) * pitch_acc;
24
25    // 5. Rumbo absoluto del magnetómetro con tilt compensation
26    const float yaw_mag = calcular_yaw_mag(m, orientacion.roll, orientacion.pitch);
27
28    // 6. Fusión Yaw considerando la discontinuidad circular [-PI, PI]
29    const float diff_yaw = normalizar_angulo(yaw_mag - yaw_gyro);
30    orientacion.yaw = normalizar_angulo(yaw_gyro + (1.0f - ALPHA) * diff_yaw);
31
32    return orientacion;
33  }
```

Código 10: Actualización en cada muestra.

Para roll y pitch, el ángulo de referencia del acelerómetro se calcula con `atan2` a partir de las componentes de la aceleración. Se aplica (1), donde la integración del giroscopio parte del ángulo estimado en la iteración anterior:

$$\theta_k = \alpha(\theta_{k-1} + \omega_k \Delta t) + (1-\alpha)\theta_{\text{acc},k} \tag{1}$$

El yaw se obtiene del rumbo magnético, como indica la (2). Con los valores de roll y pitch ya fusionados se proyecta el vector magnético sobre el plano horizontal y se calcula el rumbo con `atan2`. La fusión con el giroscopio se hace sobre la diferencia angular, normalizada al rango $[-\pi, \pi]$, para evitar saltos cuando el ángulo cruza esos límites. El magnetómetro aporta datos en cuentas crudas, lo cual es suficiente porque el rumbo depende solo de la dirección del vector. Los offsets de hard-iron están en cero a la espera de la calibración.

$$\begin{aligned} \psi_{g,k} &= \psi_{k-1} + \omega_{z,k}\Delta t \\ \psi_k &= \psi_{g,k} + (1-\alpha)\ \operatorname{norm}\big(\psi_{m,k} - \psi_{g,k}\big) \end{aligned} \tag{2}$$

donde α es el coeficiente de ponderación y $\operatorname{norm}(\cdot)$ lleva el ángulo al rango $[-\pi, \pi]$.

Las pruebas de transmisión previas a incorporar el magnetómetro confirmaron que el firmware calcula y actualiza los ángulos junto con las mediciones del sensor, sin errores I²C ni interrupciones agrupadas. Resta repetir la verificación con el magnetómetro activo, validar los

ángulos frente a orientaciones conocidas, calibrar el magnetómetro y medir la deriva residual. Además, el algoritmo integra cada eje de forma independiente, por lo que deberá evaluarse su comportamiento ante rotaciones tridimensionales combinadas.

4.6. Transmisión ESP32 a PC

La transmisión validada hasta la fecha se realiza por comunicación serie USB a 115.200 baudios. El ESP32 envía registros en formato CSV, uno por cada muestra, con el tiempo, las aceleraciones, las velocidades angulares y los tres ángulos estimados. Cada línea tiene el formato `tiempo, Ax, Ay, Az, Gx, Gy, Gz, roll, pitch, yaw`. Aún no se incluyen las mediciones del magnetómetro.

Existen además archivos preliminares para crear un punto de acceso Wi-Fi en el ESP32 y transmitir los datos mediante UDP. Sin embargo, esta vía todavía no se integró ni se ensayó con el sistema completo, por lo que la comunicación inalámbrica no se considera completada.

4.7. Otros avances

Si bien no estaba planificado para este período, se desarrolló un visualizador experimental con JavaScript y Three.js, que recibe los datos del ESP32 mediante la API Web Serial y representa la orientación estimada sobre un modelo tridimensional. En una primera versión el filtrado se realizaba en JavaScript, a partir de las mediciones crudas, posteriormente se trasladó al ESP32, de modo que el visualizador recibe directamente los ángulos calculados por el firmware.

Se verificó que el flujo de ángulos del firmware llega al visualizador, pero todavía debe confirmarse que su correspondencia con los ejes de la última versión del modelo sea correcta.

4.8. Estado general

El avance de cada actividad hasta la fecha se resume en la Tabla 7.

Tabla 7: Síntesis del avance experimental.

Actividad	Estado
Montaje sobre el guante	Pendiente
Lectura I²C y DATA_READY a 50 Hz	Probado con MPU6050
Lectura del magnetómetro (QMC5883L)	Implementado, sin calibrar
Filtro complementario en ESP32	Implementado, validación pendiente
Lectura de orientación del DMP	Probado con MPU6050 (verificación visual)
Calibración estática del giroscopio	Ensayo inicial
Transmisión CSV por USB	Probado
Visualizador Three.js	Prototipo de prueba
Comunicación Wi-Fi	Archivos preliminares, sin ensayar
Puesta en marcha del MPU9250	Pendiente

La próxima etapa incluirá, además de las tareas planificadas inicialmente para ese período, la puesta en marcha del MPU9250 y la integración de la comunicación inalámbrica, que no se llegó a completar para este informe de avance.

5. Documentación relacionada

5.1. Registro del prototipo

La Figura 4 muestra el montaje físico del prototipo experimental, compuesto por el ESP32, el MPU6050 y el magnetómetro QMC5883L, conectados según lo detallado en la Tabla 3. Esta imagen corresponde al sistema de adquisición descrito en Comunicación I²C y lectura de datos crudos de la sección Montaje de hardware.

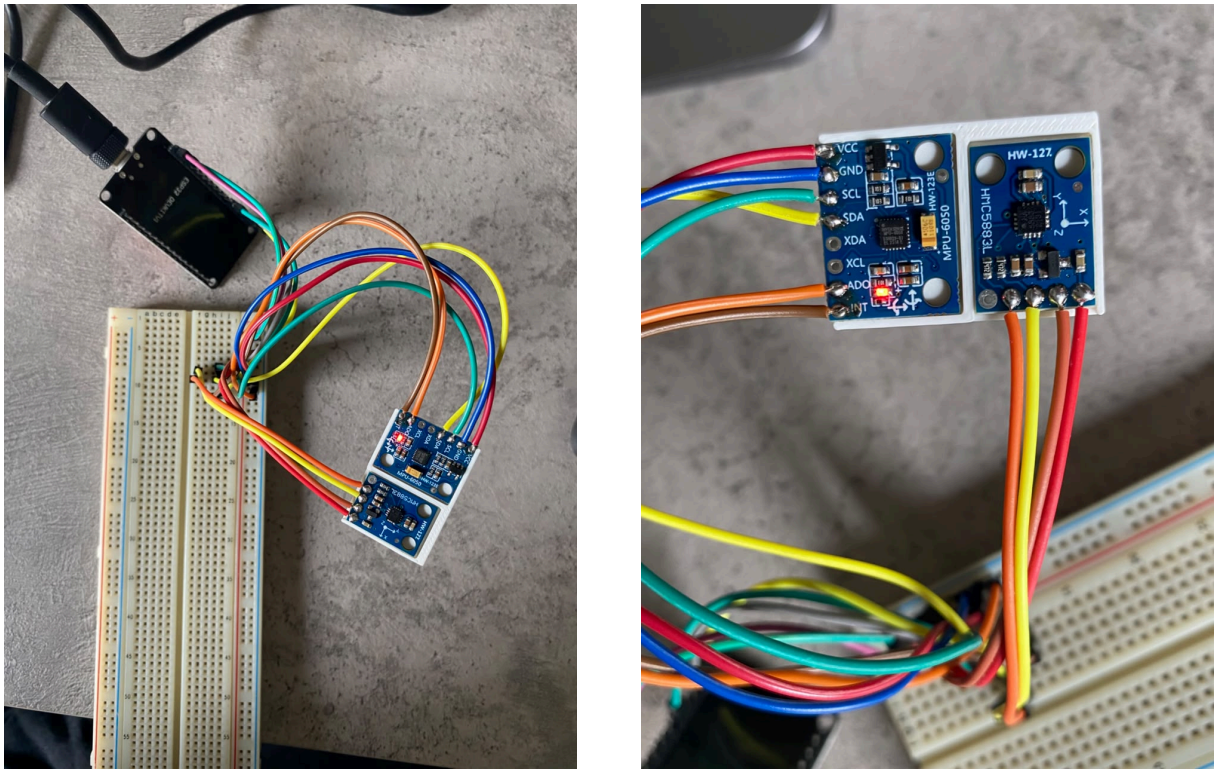


Figura 4: Prototipo completo para las pruebas con MPU6050 y magnetómetro

La Figura 5 presenta una captura del visualizador experimental desarrollado con JavaScript y Three.js, que recibe por Web Serial los ángulos calculados por el ESP32 y los representa sobre un modelo tridimensional. Corresponde a la sección Otros avances y recibe los datos descritos en Transmisión ESP32 a PC.

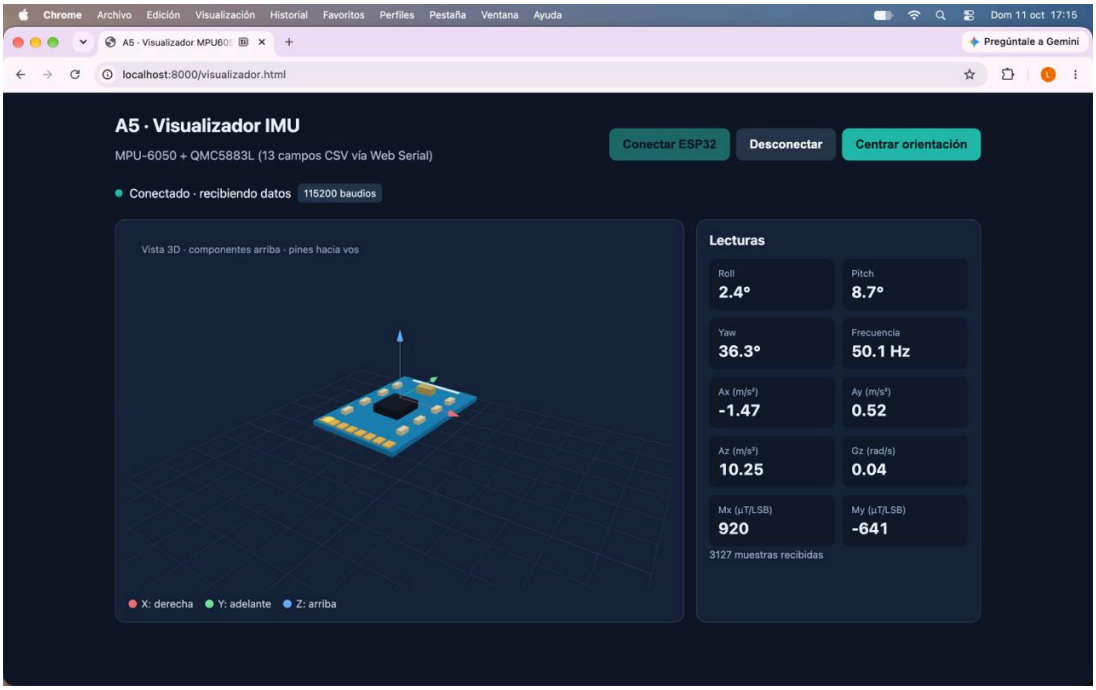


Figura 5: Captura del visualizador experimental con Three.js.

5.2. Links de la entrega

El código fuente está disponible en el repositorio del proyecto, la bitácora de trabajo en la wiki del repositorio y el video de demostración en Google Drive.